Engineering Portfolio

Dylan Piwko

B.S.E. Aerospace Engineering, University of Michigan

Expected Graduation: May 2030



My Terrier-Sandhawk two-stage rocket at liftoff

dpiwko@umich.edu

linkedin.com/in/dylan-piwko

Table of Contents

Page 2: Kestrel Flight Computer
Designer, Personal Project · KiCad, PCB design, embedded C++, RF

Page 4: NASA Ames Research Center, Aeropulse
Concept Design Intern · Systems design, UAS routing, airspace integration

Page 6: Terrier-Sandhawk Two-Stage Rocket
Builder and Flyer, Personal Project · RockSim, electronic staging, failure analysis

Page 8: EUROCONTROL PRC Data Challenge
Team Member, Embry-Riddle team · Python, LightGBM, CatBoost, SHAP

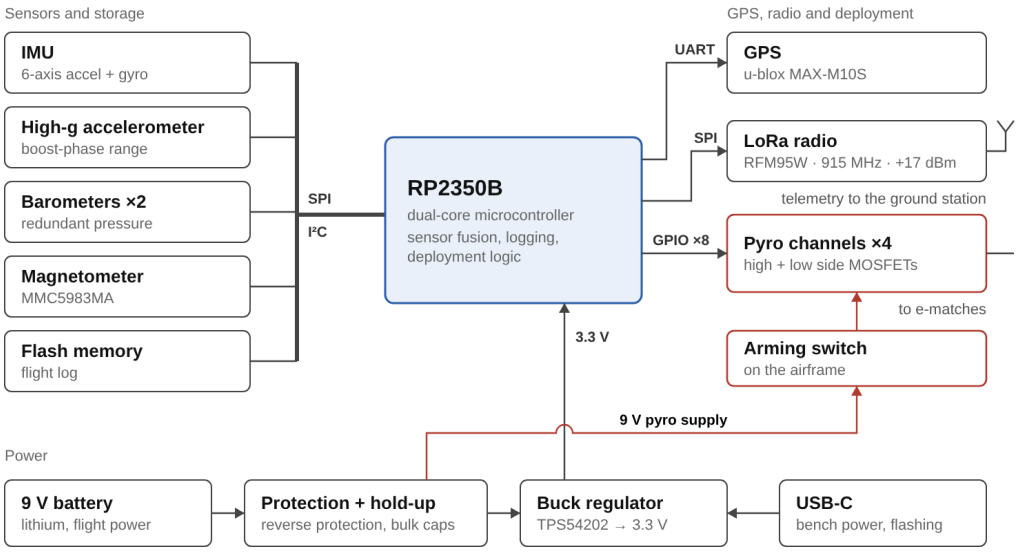
Kestrel Flight Computer (September 2026 – Present)

Designer, Personal Project

Overview: Kestrel is a flight computer I'm designing for my 4-inch Wildman Punisher 4, the rocket I'm flying for my NAR Level 2 certification in January 2027. It logs flight data, sends live telemetry and is built to eventually run dual-deployment recovery, including on a later L2200G flight that simulates to about 14,800 ft and Mach 1.7. The four-layer, 100 × 40 mm board has 145 parts and is being made by PCBWay through their Future Innovators sponsorship.

Individual Responsibilities:

- Designed the schematic and four-layer PCB in KiCad
  - RP2350B microcontroller with an IMU, high-g accelerometer, two barometers, magnetometer, u-blox GPS, 915 MHz LoRa radio and flash logging
  - Four pyro channels switched on the high and low side behind a physical arming switch
  - 9 V input with reverse protection, regulated to 3.3 V by a TPS54202 buck converter
- Verified the design before ordering, since my budget allows one board revision
  - Checked every part pin by pin against its datasheet and Raspberry Pi's reference design
  - Reached zero KiCad ERC/DRC errors and fixed the items an outside review flagged
- Developed bring-up firmware and a desktop ground station with a live map and pyro continuity
- Planned a staged flight program that keeps commercial altimeters in charge of recovery until Kestrel passes ground and flight tests



System block diagram. The pyro channels draw power through the arming switch.

Kestrel Flight Computer: Design Details

Pyro channel safety, verification and flight plan

Each pyro channel switches the e-match on both its high side and its low side, behind a physical arming switch on the airframe. A charge can only fire when the switch is closed and the processor turns on both transistors. Pull-down resistors hold every gate off while the processor boots, and a small sense current through each e-match shows continuity without firing it.

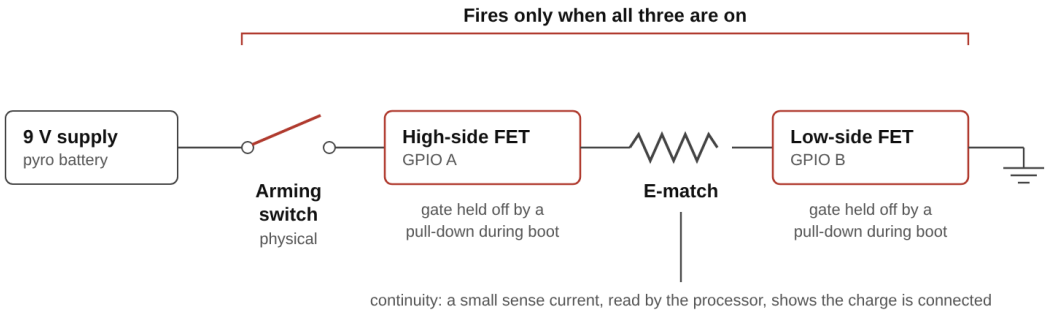


Figure 1. *One of four pyro channels. Three independent conditions must all be true to fire.*

Because I can only afford one board revision, I checked each subsystem against a datasheet or reference design before ordering, and planned a specific test to prove it once the boards arrive.

Subsystem	Checked against	Proven at bring-up by
Power	TI's TPS54202 3.3 V reference values; USB-only start-up margin	3.25–3.35 V at the test point
Processor	All 80 RP2350B pins against the pin table and SparkFun's production board	Enumerates over USB
Sensors and GPS	Each pinout and SPI mode against its datasheet; u-blox integration manual	Chip ID self-test; 3-D GPS fix
Radio	Pinout; link budget of about 29 dB margin at 10 km	Ground-station range test
Pyro	MOSFET ratings, gate drive, boot pull-downs	Dummy-load tests, armed and disarmed
Layout	Zero ERC/DRC errors; outside review; PCBWay engineering review	Visual inspection of both assembled boards

On the first flight Kestrel only logs and transmits, while a Featherweight Blue Jay and an EasyMini handle recovery. Before it controls a parachute, its would-fire times from flight 1 have to match the commercial altimeters, it has to fire correctly on replayed simulated L2200G flights and never during the Mach transition, and it has to pass vacuum-chamber, inert-load and ground ejection tests.

NASA Ames Research Center (June 2025 – August 2025)

Concept Design Intern, Aviation Systems Division

Overview: I was one of 36 students selected from about 800 applicants for the Aviation Systems Division's Concept Design Experience. Our team of six designed Aeropulse, a network of hybrid VTOL drones and stair-climbing robots that moves organs and urgent medical supplies between hospitals without hand-offs. The study was published as NASA Technical Memorandum NASA/TM-20250009029.

Individual Responsibilities:

- Analyzed drone routing and urban airspace integration (see next page)
  - Pre-flight planning around no-fly zones, altitude limits, weather and UTM clearance
  - In-flight detect-and-avoid, re-routing and fallback options for failures
  - Operations under 400 ft AGL under Part 107 with BVLOS waivers
- Evaluated trade-offs between the drone and robot segments of the system
- Presented the design to 50+ NASA staff and co-authored the Technical Memorandum



The Aeropulse concept: a robot carries the payload out of Hospital A, rides a hybrid VTOL drone, and delivers it inside Hospital B



(L to R): Modeled delivery route from San Francisco General to Santa Clara Valley Medical Center (map data © Google), the team's sub-scale 3D-printed robot prototype

Links:

- https://ntrs.nasa.gov/citations/20250009029

Aeropulse: Drone Route Planning

My focus area: how the drone plans a legal route and adapts in flight

Before each mission the drone plans its route from Hospital A to Hospital B. The inputs are the start and end coordinates, altitude limits, the weather forecast, no-fly zones and UTM airspace availability. These are compiled into a 3-D occupancy grid, the global path is planned around obstacles and restricted airspace at the lowest cost, and the drone requests emergency priority clearance through UTM.

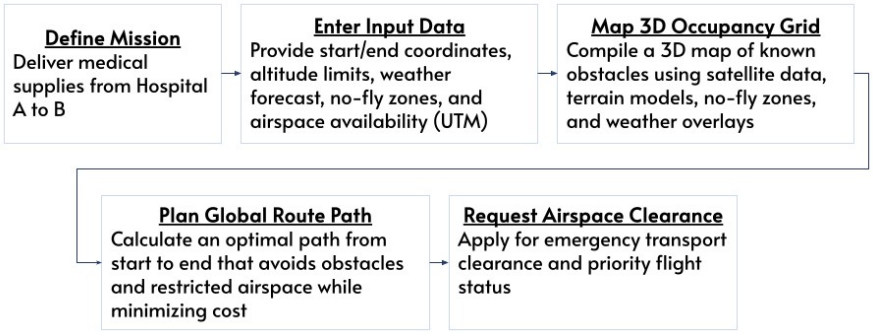


Figure 1. *Pre-flight route optimization, from the memo.*

In flight, the drone keeps its map updated and fuses GPS, IMU, odometry, camera and LiDAR data with SLAM and a Kalman filter to track where it is. If the weather shifts, another aircraft conflicts or a new restriction appears, it replans and resubmits the route through UTM. If it can't continue safely, it can loiter, land at a low-risk site, return to base or deploy a parachute.

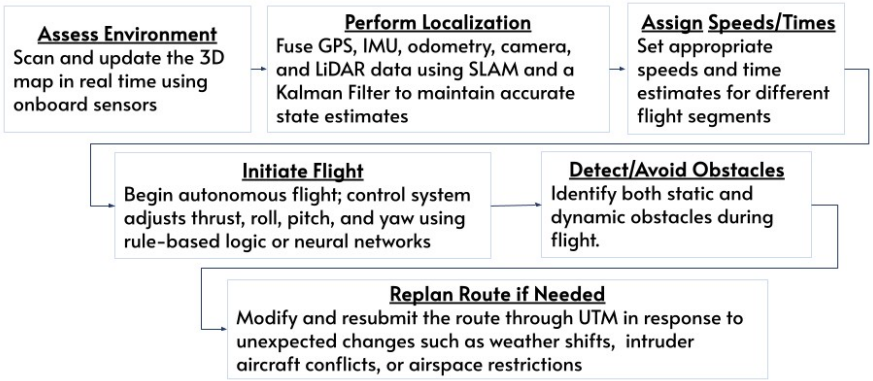


Figure 2. *In-flight route optimization, from the memo.*

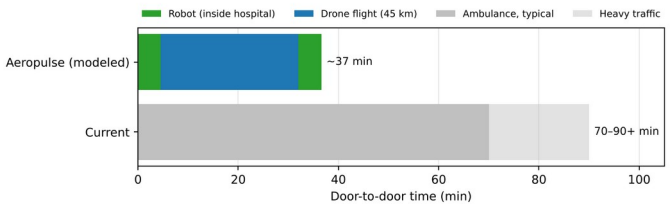


Figure 3. *Modeled heart transfer, San Francisco General to Santa Clara Valley Medical Center: about 37 min against 70 to 90+ min today.*

Terrier-Sandhawk Two-Stage Rocket (January 2025)

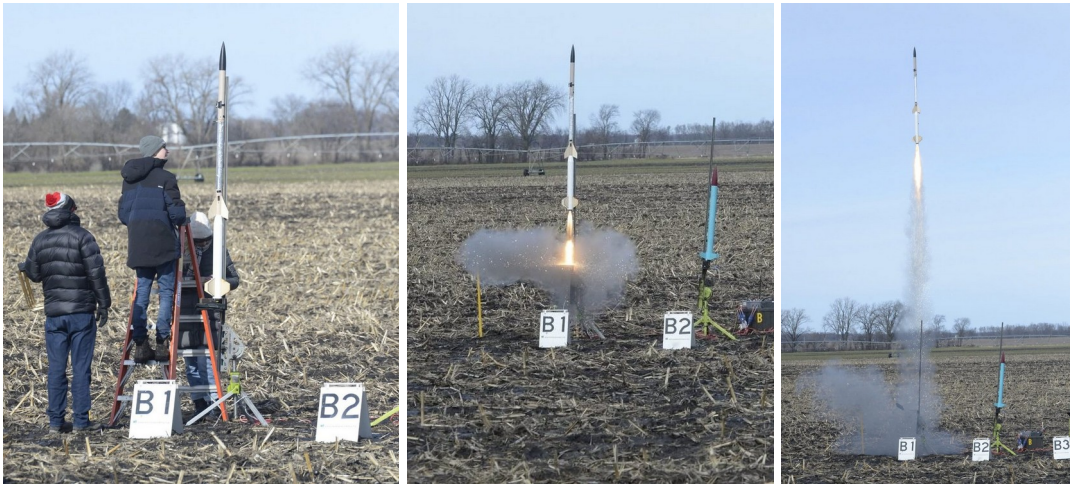
Builder and Flyer, Personal Project

Overview: My first two-stage high-power rocket is a LOC Precision 2.26" Sandhawk sustainer on a Terrier booster, flown on a Cesaroni I297 Skidmark in the booster and an H180 Skidmark in the sustainer. It is 76.9 in long and 2,078 g at liftoff. Because composite motors can't reliably light an upper stage on their own, staging is electronic: the stages separate 1.0 s after booster burnout and the sustainer lights 0.7 s later, once the booster has fallen away. I flew it at Michiana Rocketry on January 31, 2025.

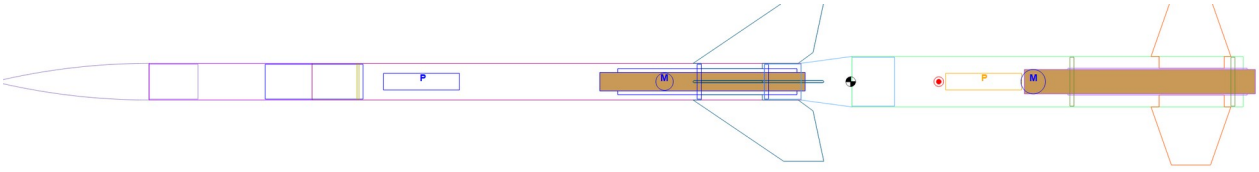
Individual Responsibilities:

- Built the rocket and its avionics for electronic separation, sustainer air-start, GPS tracking and telemetry
- Simulated the flight in RockSim to pick motors and check stability (2.40 static margin)
- Analyzed how the 0.7 s ignition delay changes the trajectory (see sample write-up)
- Investigated why the flight failed
    - The stages didn't separate properly, the sustainer never lit, and the upper stage was not recovered
    - Every flight computer ran on a 9 V battery mounted to one sled that wasn't secured well enough; the sled most likely slid down the airframe under boost acceleration and disconnected both batteries

Lesson: This flight taught me the importance of redundant avionics systems. Both batteries were on the same sled, so one mounting problem took out all of the electronics at once.



(L to R): Setting up on pad B1, liftoff on the I297 booster, climbing before staging



RockSim model: sustainer on the left, booster on the right (CG 52.5 in, CP 57.9 in from the nose)

Terrier-Sandhawk: Sample Write-Up

Effect of the sustainer ignition delay on the predicted flight

RockSim lights the sustainer at the moment the stages separate, so its prediction does not include the 0.7 s wait before sustainer ignition. To model the actual sequence, the upper stage was re-flown from separation with a two-dimensional point-mass model. The sustainer's drag-to-mass ratio was taken from RockSim's coast phase as a function of Mach number, and its ignition mass was fitted to the boost phase using the H180's certified thrust curve. Run with RockSim's own timing, the model matches RockSim's apogee within 3 ft and its peak speed within 0.3%.

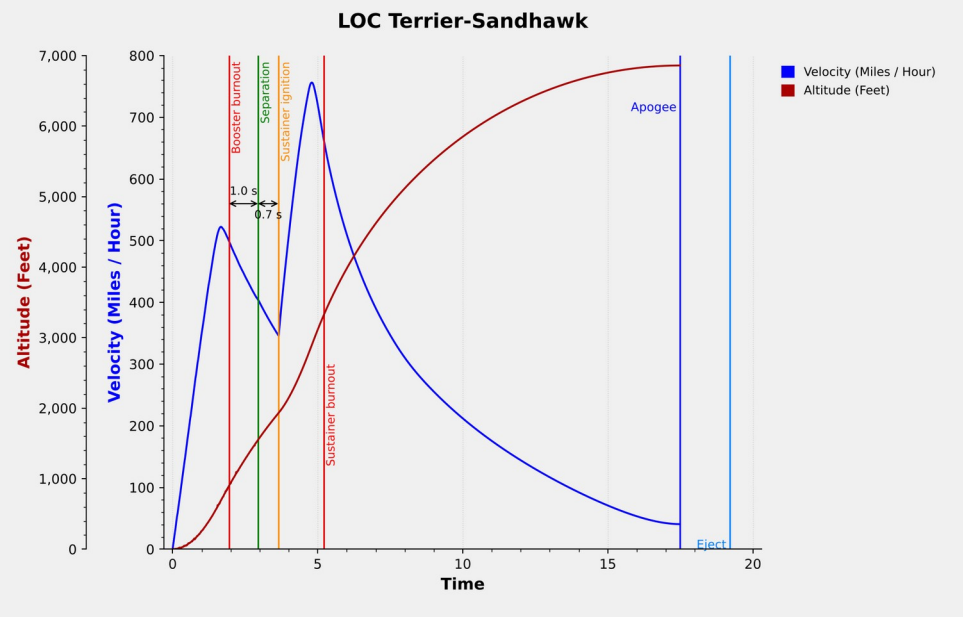


Figure 1. *Predicted flight with the 1.0 s separation and 0.7 s ignition delays, in RockSim's plot format.*

With the delay, the sustainer lights at 347 mph instead of 415 mph and peaks at 756 mph instead of 775 mph. Less of the burn happens near Mach 1, where drag is highest, so the predicted apogee rises from about 6,590 ft to 6,860 ft. A 10% change in drag moves this by about 300 ft. Because the sustainer never lit on this flight, the prediction couldn't be checked against flight data.

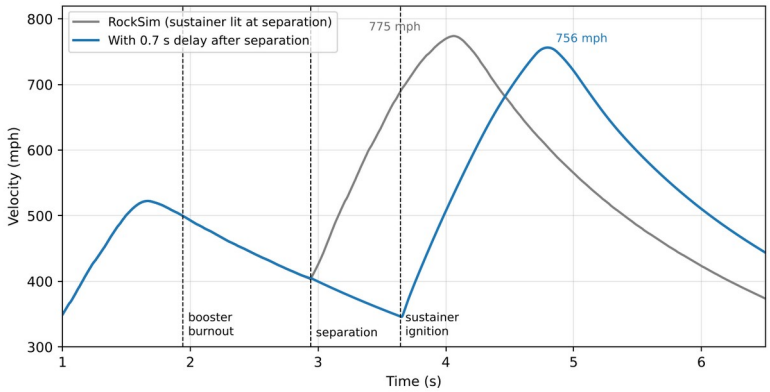


Figure 2. *Staging close-up: RockSim's run (gray) against the delayed ignition (blue).*

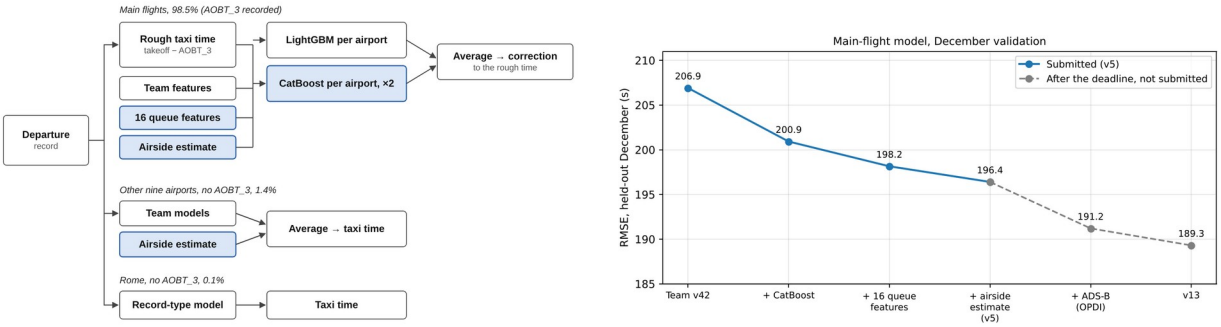
EUROCONTROL PRC Data Challenge (September 2026 – October 2026)

Team Member, Embry-Riddle Aeronautical University team "merry-llama"

Overview: EUROCONTROL's Performance Review Commission challenged teams to predict aircraft taxi-out time at ten major European airports using about 2 million 2025 departures, scored by RMSE on flights from January and July 2026. I joined an Embry-Riddle team with very little Python or machine learning experience and worked on the model for main flights, which make up 98.5% of departures.

Individual Responsibilities:

- Added CatBoost models alongside the team's LightGBM models
- Engineered 16 departure-queue features, such as same-runway takeoffs in the previous 5 to 20 minutes
- Used SHAP to analyze the 1,000 worst predictions
    - Found that 82% had a reported off-block time more than 10 minutes wrong
    - Trained an "airside" taxi-time estimate that does not depend on it
- Lowered the team's official RMSE from 276.53 s to 267.34 s; teammate Taeyun Yoo built the team's best score (265.02 s) and final pipeline on this version



(L to R): The team's model with my additions shaded blue; held-out error after each of my changes (gray points came after the deadline and weren't submitted)

Links:

- https://github.com/taeyunyoo/prc-2026-merry-llama